COSMOS DB CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. OVERVIEW & SETUP

Installation

Cosmos DB managed cloud service

```
# Install CLI
aws configure  /  az login  /  gcloud auth lo...
# Set region
aws configure set region us-east-1
```

Use IAM roles in production, not access keys

04. CONFIGURATION

Workflow

```
# AWS CLI config: ~/.aws/credentials
[default]
aws_access_key_id = AKIA...
aws_secret_access_key = ...
region = us-east-1
output = json
```

Prefer environment variables or instance profiles over config files

02. AUTHENTICATION & IAM

Architecture

IAM Roles, Policies, and Least Privilege

```
# Assume role
aws sts assume-role --role-arn arn:aws:iam::1...
--role-session-name session
# Attach policy to role
aws iam attach-role-policy --role-name MyRole...
--policy-arn arn:aws:iam::aws:policy/ReadOnly...
```

05. MONITORING & ALERTS

CRUD API

Enable CloudWatch / Azure Monitor / Cloud Monitoring

```
# Create CloudWatch alarm
aws cloudwatch put-metric-alarm \
--alarm-name HighCPU \
--comparison-operator GreaterThanThreshold \
--threshold 80
```

Set dashboards for key metrics before production launch

03. CORE FEATURES

YAML / Config

Key capabilities and use cases for this service

```
# Create resource
aws <service> create-<resource> --name MyReso...
# List resources
aws <service> list-<resources>
# Describe resource
aws <service> describe-<resource> --name MyRe...
```

06. SECURITY BEST PRACTICES

Integration

Enable encryption at rest and in transit

```
# Enable server-side encryption
--sse-algorithm aws:kms
--kms-key-id arn:aws:kms:...
```

Rotate credentials regularly
Enable CloudTrail / activity logs for audit
Use VPC for network isolation

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. SDK USAGE

Hardening

```
# Python (boto3)
import boto3
client = boto3.client('s3', region_name='us-e...
client.upload_file('file.txt', 'my-bucket', '...
# Node.js
const AWS = require('@aws-sdk/client-s3')
```

Use SDK over direct HTTP for automatic retry/signing

08. CLI COMMANDS

Observability

```
aws <service> help
aws <service> list-<resources> --query '...' ...
aws <service> create-<resource> --cli-input-j...
aws <service> delete-<resource> --name ...
aws cloudformation deploy --template-file tem...
```

09. INTEGRATION PATTERNS

Optimization

Event-driven: trigger functions on service events

```
# SNS  Lambda  SQS pattern
# API Gateway  Lambda  DB
```

Use managed services over self-hosted where possible
Infrastructure as Code: Terraform/CDK/CloudFormation
Multi-region for disaster recovery

10. PRICING & COST OPT

Unit Tests

Understand pricing model: per request, per GB, per hour
Use Reserved Instances / Committed Use for 30-70% savings
Enable Cost Explorer / budget alerts
Right-size resources regularly

```
# Tag resources for cost allocation
aws <service> tag-resource --tags Project=mya...
```

11. DEPLOYMENT PATTERNS

Commands

Blue/Green: shift traffic between environments
Canary: gradual traffic shift with rollback

```
# Infrastructure as Code
terraform init && terraform plan && terraform...
# Or CloudFormation
aws cloudformation deploy --stack-name myapp \
--template-file infra.yaml
```

12. COMMON GOTCHAS

Warning

Never commit credentials to git use secrets manager
Check service quotas before scaling
Test in staging before production deployments
Enable MFA for root account and admin users
Review IAM policies regularly remove unused permissions